

UNIVERSITÀ DEGLI STUDI DI ROMA “TOR VERGATA”

Macroarea di Ingegneria

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

INGEGNERIA DEL SOFTWARE E PROGETTAZIONE WEB (ISPW)

CIBOAMICO

*PLATFORM FOR MANAGING FOOD AT HOME, REDUCING WASTE AND
OPTIMISING THE SHOPPING EXPERIENCE*

SUPERVISORS:

PROF. DAVIDE FALESSI

PROF. GUGLIELMO DE ANGELIS

AUTHOR:

MICHELE DAMIANO

STUDENT ID: 0324516

MICHELE.DAMIANO@STUDENTS.UNIROMA2.EU

ACADEMIC YEAR 2025/2026

SUBMISSION DATE: AUGUST 13, 2026

Contents

1	Software Requirements Specification	3
1.1	Introduction	3
1.1.1	Purpose of the document	3
1.1.2	System overview	3
1.1.3	Hardware and software requirements	4
1.1.4	Related systems	4
1.2	User Stories	5
1.3	Functional Requirements	5
1.3.1	Glossary of Domain Terms	6
1.4	Use Cases	6
1.4.1	Use case diagram	6
1.4.2	Internal Steps	6
2	Storyboards	8
2.1	Authentication and Personal Area	8
2.1.1	Login	8
2.1.2	Home (Customer)	8
2.2	Purchases and Marketplace (UC-04 Order a Product)	9
2.2.1	Marketplace (Product Catalog)	9
2.2.2	Order a Product	10
2.2.3	Payment (step 6)	11
3	Design	13
3.1	Class Diagram	13
3.1.1	VOPC (Analysis)	13
3.1.2	Design Patterns	13
3.2	Activity Diagram	17
3.3	Sequence Diagram	18
3.4	State Diagram	18
4	Testing	19

5	Exceptions	21
5.1	Wrapping	21
5.2	Payment	21
5.3	Messages	22
6	Persistence	23
6.1	DB persistence layer (MySQL/JDBC)	23
6.2	FS persistence layer (File System)	23
6.3	DEMO persistence layer (In-Memory)	24
7	Video and Links	25
7.1	GitHub Repository	25
7.2	SonarCloud	25
7.3	Demonstration video	25

Chapter 1

Software Requirements Specification

1.1 Introduction

1.1.1 Purpose of the document

This document defines the software requirements of **CiboAmico**, the final project of the ISPW course (University of Rome “Tor Vergata”, A.Y. 2025/2026, Prof. Davide Falessi, Prof. Guglielmo De Angelis).

The purpose is to state what the system shall do in a clear and verifiable way, leaving room for design and testing. The document serves both developers, who must keep the Boundary-Control-Entity (BCE) architecture, and the evaluators, who compare the functional flows against the initial requests.

1.1.2 System overview

CiboAmico is a desktop JavaFX application that helps manage food at home, avoid waste and optimise the shopping experience by connecting the user with local vendors. The system follows the Boundary-Control-Entity (BCE) pattern and involves four actors: Customer, Vendor, Nutritionist and Administrator. Two external systems are reached through an Adapter: OpenFoodFacts (reading nutritional data from the barcode) and JakartaMail (sending notifications). Persistence is interchangeable at runtime between a *Demo* (in-memory) and a *Full* (MySQL via JDBC) mode.

The system has three functional areas:

- **Pantry:** the Customer keeps track of the food at home, sees the expiry dates and is warned before the food goes bad;
- **Recipes:** the system proposes recipes compatible with the available ingredients, substituting the missing ones;

- **Marketplace:** the Vendors publish products, the Customers place direct orders (one buyer, one vendor) and the Administrator approves vendors and recipes.

1.1.3 Hardware and software requirements

The application is cross-platform thanks to the JVM.

Software requirements:

- Java 17+
- JavaFX 17+ (for the desktop interface)
- Maven 3.9+ (build tool)
- MySQL Server 8.0+ (*Full* mode, via JDBC)
- Draw.io (for creating the UML diagrams)
- IDE: IntelliJ IDEA (with EclEmma for coverage) or Eclipse

Hardware requirements:

- CPU: Dual-Core 2.0 GHz 64-bit
- RAM: 2 GB (4 GB recommended in *Full* mode with a local MySQL server)
- Disk space: 1 GB
- Screen: 1024 × 768
- Operating system: Windows, macOS, Linux (compatible with Java 17+)

1.1.4 Related systems

Two existing systems cover only in part what CiboAmico does.

System 1: Too Good To Go

Pros:

- Wide network of partners with geolocated offers in real time.
- Actively reduces commercial waste by offering food at reduced prices.

Cons:

- It does not track the home pantry: after the purchase it does not help manage the expiry dates.

- The purchase is a surprise *Magic Box*: the user does not choose the single products nor plan recipes or diets.

System 2: SuperCook

Pros:

- Powerful matching: it quickly finds many recipes from the inserted ingredients.
- Large database with filters by dish type and intolerances.

Cons:

- It has no integrated purchase channel: the user must buy the missing ingredients by themselves.
- It does not automatically substitute the ingredients, excluding recipes that would be possible with small variations.

CiboAmico aims to combine pantry tracking, recipe generation with substitutions and direct purchase from local producers in a single platform, providing features dedicated to customers, vendors, nutritionists and administrators while keeping a modular and extensible architecture.

1.2 User Stories

1. **US-1:** As a Customer, I want to purchase products from verified local vendors, so that I can support nearby businesses and receive my items faster.
2. **US-2:** As a Customer, I want to apply a promotional voucher code during checkout, so that I can reduce the total cost of my order and save money.
3. **US-3:** As a Customer, I want to pay for my order securely using my credit card, so that I can complete my purchase immediately without handling physical cash.

1.3 Functional Requirements

1. **FR-1:** The system shall display the catalog of the products published by approved vendors, with price and available quantity.
2. **FR-2:** The system shall apply a promotional voucher to the current order when the code is valid and belongs to the vendor of the selected product.
3. **FR-3:** The system shall register a purchase order by associating the customer with the vendor and decreasing the product stock by the ordered quantity.

1.3.1 Glossary of Domain Terms

Catalog : set of products published by approved vendors and made available for purchase in the marketplace.

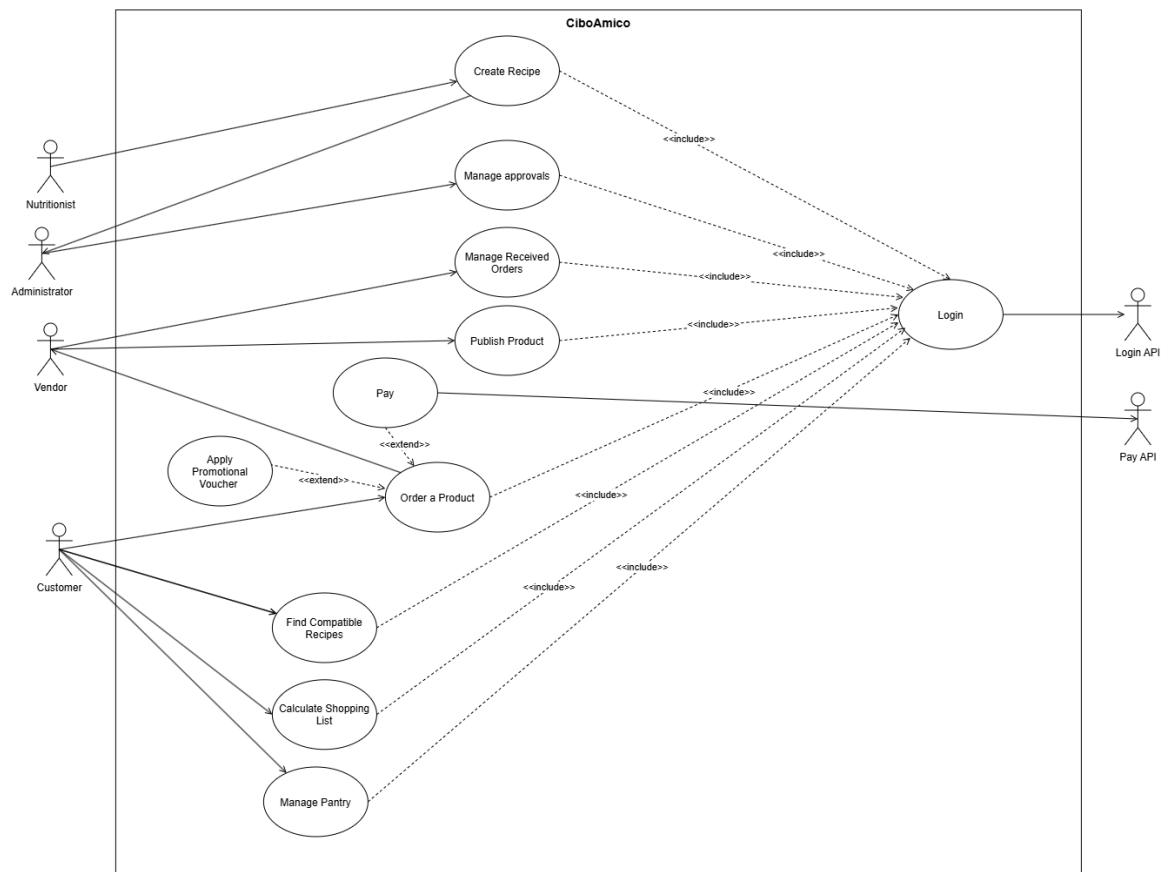
Promotional voucher : discount code associated with a specific vendor, valid within a time window and applicable to an order of that vendor.

Direct order : transaction that associates a single buying customer with a single vendor, without passing through a shared cart or intermediaries.

Product availability : quantity of a product currently sellable, which can never be negative and is updated after every purchase.

1.4 Use Cases

1.4.1 Use case diagram



1.4.2 Internal Steps

Order a Product

Main Flow:

1. The customer accesses the marketplace via Login.
2. The system displays the available products with price and remaining stock.
3. The customer selects a product from the catalog and starts the checkout.
4. The system loads the selected product into the current order and computes the total amount.
5. The customer confirms the purchase.
6. The system requests the authorization for the debit of the computed amount from the Payment Service.
7. The system registers the order in the “CREATED” state, decreases the remaining stock of the product and notifies the vendor.

Preconditions: the customer is authenticated; at least one product is available in the catalog.

Alternative Flow (Extensions):

- **3a. Product No Longer Available:** the selected product is not present in the catalog anymore; the system displays an error message.
- **4a. Promotional Voucher Request:** the customer requests to apply a promotional voucher to the current order; the system validates the temporal validity of the voucher and its association with the vendor of the selected product, recalculates the total amount applying the discount and resumes from step 5.
- **6a. Payment Rejected:** the Payment Service returns a negative outcome; the system cancels the current order and displays a failure message, returning the flow to step 4.

Chapter 2

Storyboards

2.1 Authentication and Personal Area

2.1.1 Login

The login screen allows the user to authenticate by inserting their credentials (email and password).

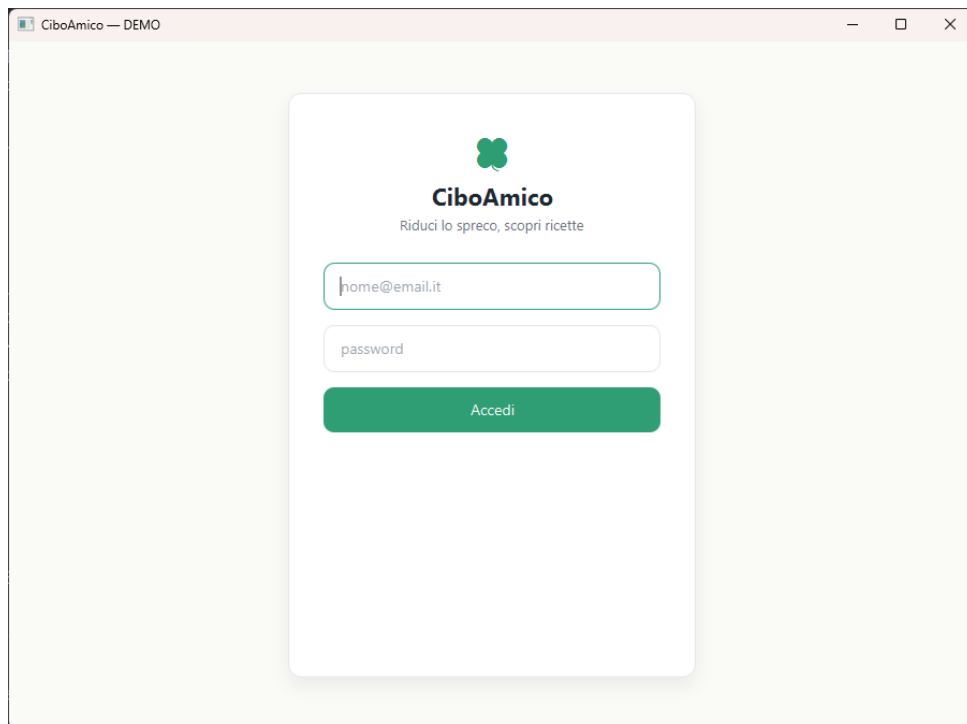


Figure 2.1: Authentication screen of the system.

2.1.2 Home (Customer)

Once authentication is completed, the Customer reaches the main dashboard (Home). From here they can access the application features: Recipes, Inventory, Shopping List and

Marketplace (UC-04 Order a Product).

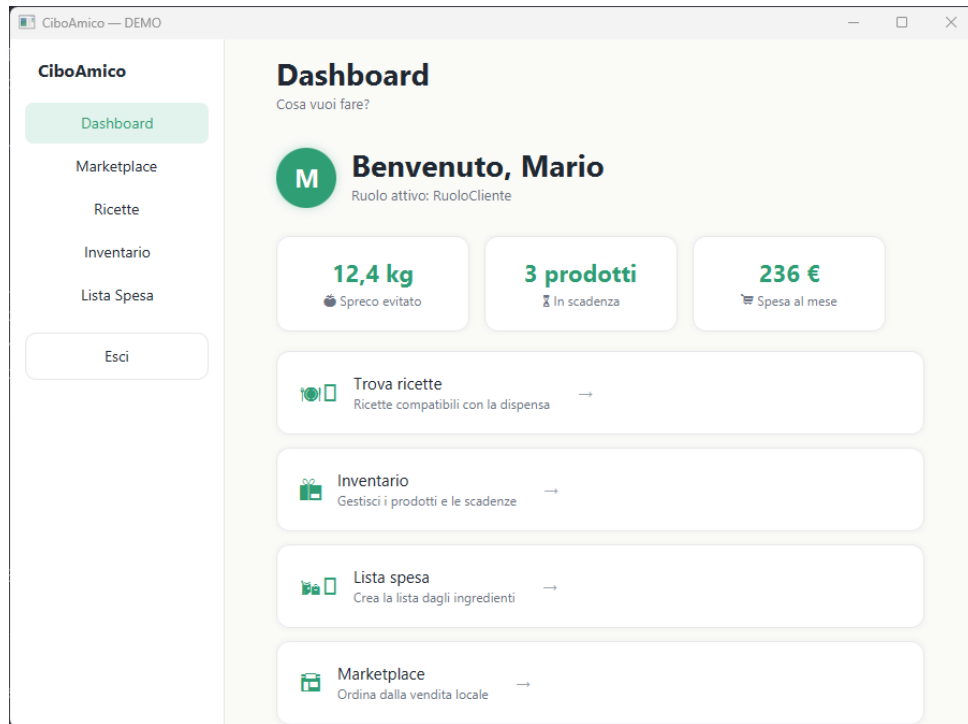


Figure 2.2: Main dashboard: access to the application features.

2.2 Purchases and Marketplace (UC-04 Order a Product)

2.2.1 Marketplace (Product Catalog)

The area dedicated to direct purchase shows the catalog of local vendor products. The side panel allows the user to select a product to order and, optionally, a promotional voucher.

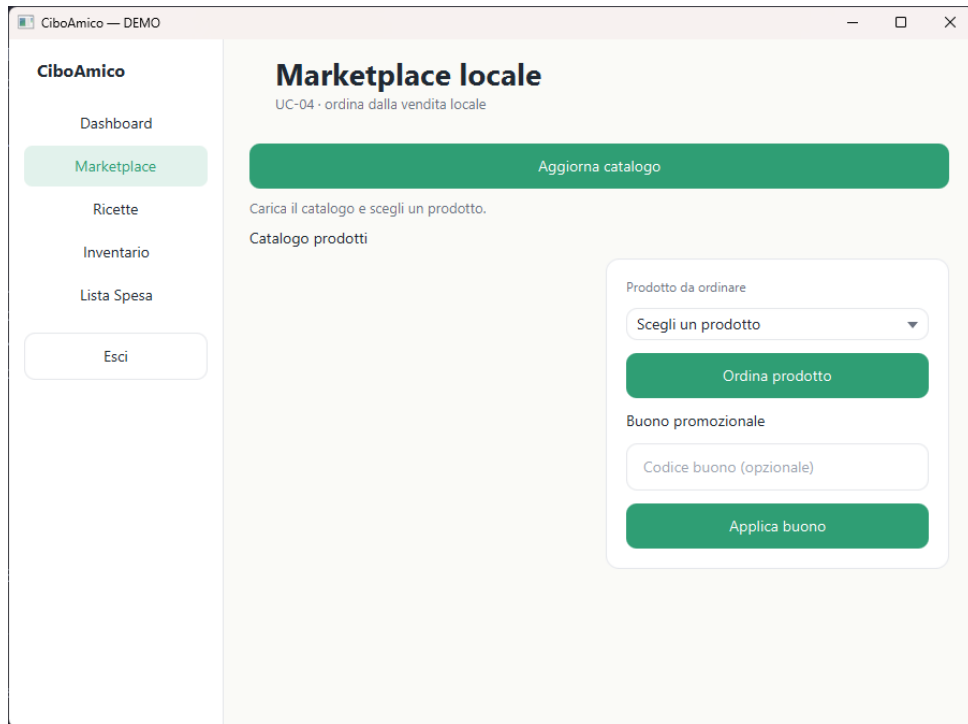


Figure 2.3: Marketplace: local product catalog and purchase panel.

2.2.2 Order a Product

After selecting a product from the catalog and loading the list (button *Refresh catalog*), the Customer chooses the product in the *Product to order* panel. If they insert a voucher code and press *Apply voucher*, the system validates the voucher and recalculates the total by applying the discount (extension *Order a Product*); by pressing *Order product* the flow proceeds.

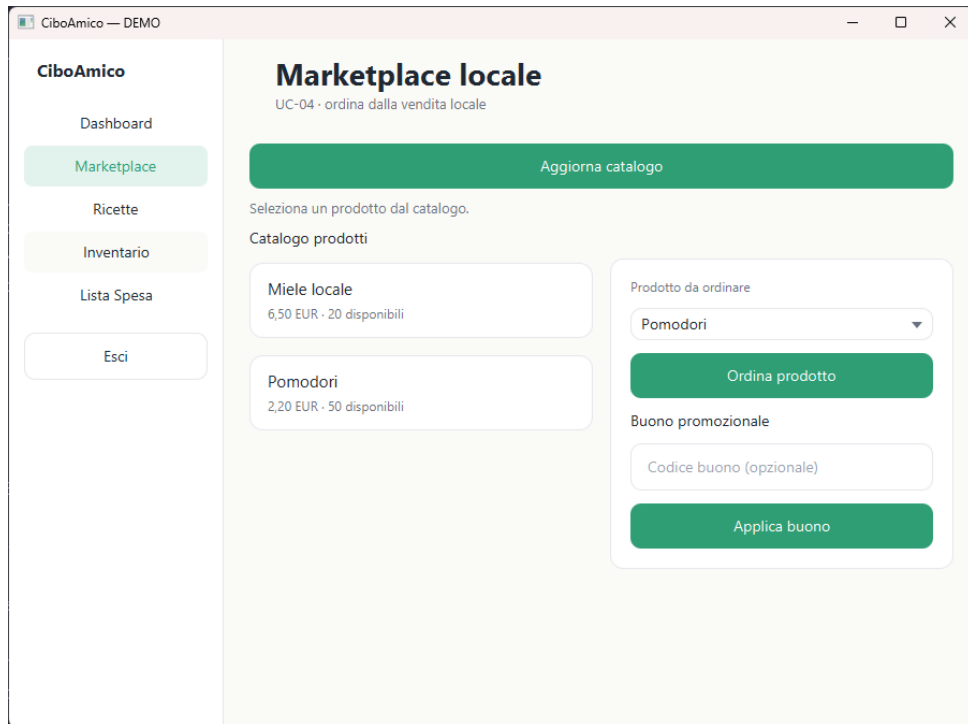


Figure 2.4: Purchase screen with the selected product and the promotional voucher field.

2.2.3 Payment (step 6)

Once the order is confirmed, the interface moves to the payment screen: the user inserts the card data (number, holder, expiry date and CVV) and authorizes the debit; the outcome is shown inline in the same view, as required by extension **6a** (rejected payment) of the use case.

The screenshot shows a web application window titled "CiboAmico — DEMO". On the left is a sidebar menu with the following items: "Dashboard", "Marketplace" (highlighted in green), "Ricette", "Inventario", "Lista Spesa", and "Esci". The main content area is titled "Pagamento" and includes the subtitle "UC-04 - autorizzazione all'addebito". Below this, it shows a summary: "Riepilogo: Pomodori — 2,20 EUR". The form contains several input fields: "Numero carta", "Intestatario", "Scadenza (MM/AA)", and "CVV". At the bottom of the form are two large green buttons labeled "Paga" and "Annulla".

Figure 2.5: Payment screen (card data and debit authorization).

Chapter 3

Design

3.1 Class Diagram

3.1.1 VOPC (Analysis)

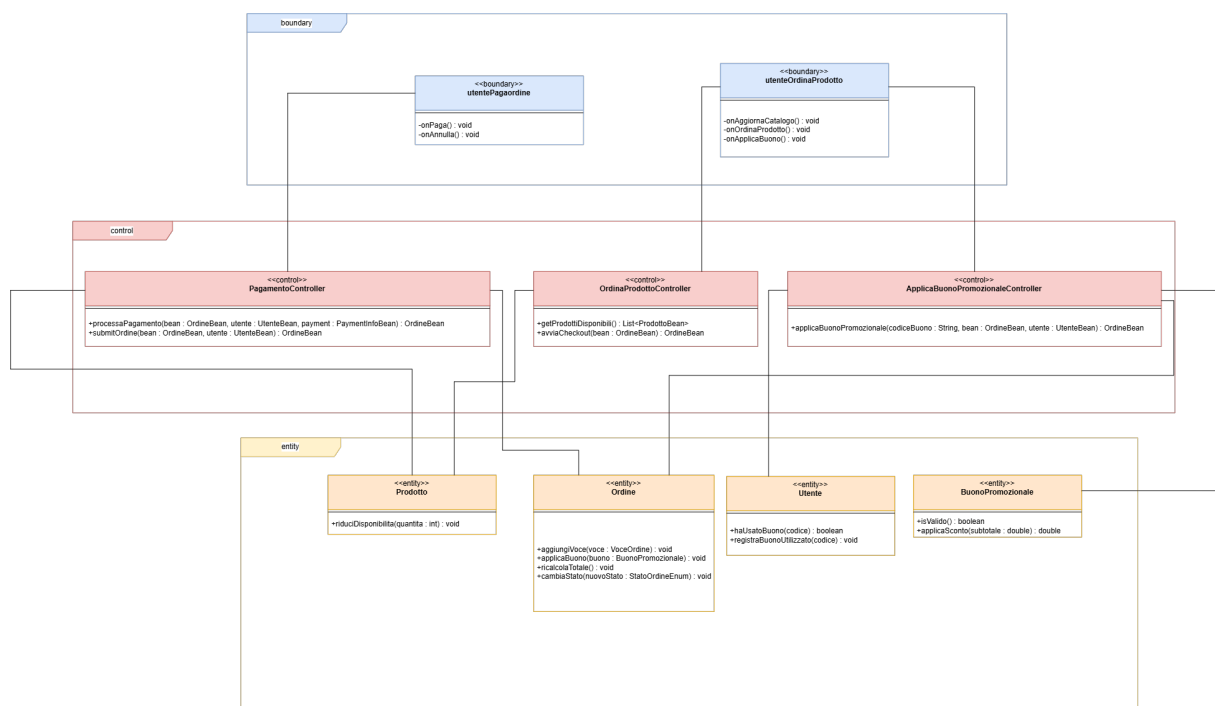


Figure 3.1: VOPC of UC-04: Boundary (View), graphic controller, Facade, application controllers and Entity.

3.1.2 Design Patterns

The design level introduces the interfaces, the Java types, the exceptions and the GoF patterns to manage extensibility and dynamic persistence (NFR-01). For each pattern, the schema and a short description of its application are shown in order.

Abstract Factory Pattern – Persistence Layer

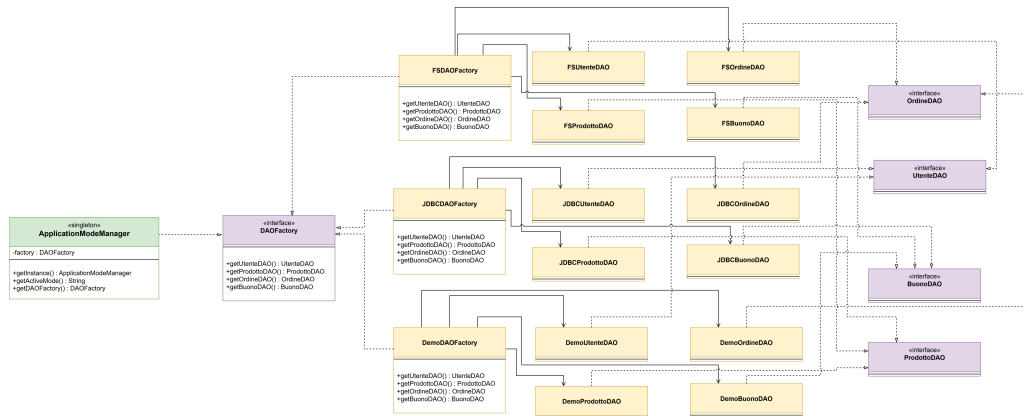


Figure 3.2: Abstract Factory of the persistence: DAO interfaces, concrete DAOs and factories.

The persistence layer is managed through the **Abstract Factory** pattern in order to guarantee flexibility in configuring the execution environment. The `DAOFactory` interface defines the contract for creating the different Data Access Objects, such as `getOrderDAO()` and `getProdottoDAO()`. According to the `persistence_type` property, the concrete factories (`JDBCDAOFactory`, `FSDAOFactory`, `DemoDAOFactory`) instantiate the appropriate storage implementation. This allows the system to switch seamlessly between a database-based, file-system-based or in-memory persistence without requiring any change in the client code.

Observer Pattern – Notification System

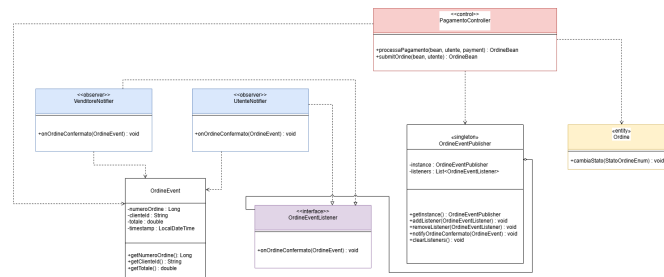


Figure 3.3: Observer of the order confirmation: singleton publisher and DTO.

The **Observer** pattern has been implemented with the aim of decoupling the order processing from the notification management logic. The `OrdineEventPublisher` class acts as a *Subject* implemented through the **Singleton** pattern, managing a list (`List`) of observers of type `OrdineEventListener`. Upon the correct submission and confirmation of the order, the publisher invokes the `onOrdineConfermato()` method on all the registered listeners (buyer and vendor), passing the read-only DTO `OrdineEvent`, never the domain

entity. This architecture guarantees loose coupling between the main business logic and its side effects, strictly respecting the *Open/Closed Principle*.

The notification is emitted by the application subsystem (the control that confirms the order) through the publisher, and not by the `Ordine` entity: the domain entity thus remains *decoupled* from the presentation layer and from the notification mechanism. The variant is the *publish-subscribe*/event-driven one of the Observer: the coupling between `OrdineEventPublisher` (Subject) and the observers is abstract (only `OrdineEventListener`), the notification is a broadcast to all the registered listeners and the exchanged data is the read-only DTO `OrdineEvent`, never the entity.

Singleton Pattern – Sessions and Mode

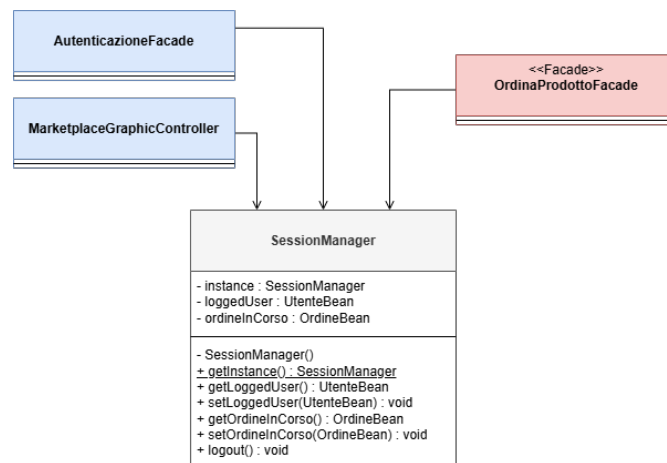


Figure 3.4: Singleton of the session and of the mode.

`SessionManager` (user session) and `ApplicationModeManager` (application mode) adopt a classic singleton with *lazy initialization* (`if (instance == null) instance = new X()`) and synchronization on `getInstance()`, in line with the reference projects: no perceivable cost in a desktop client, no SonarCloud warning suppression.

Facade Pattern – Checkout UC-04

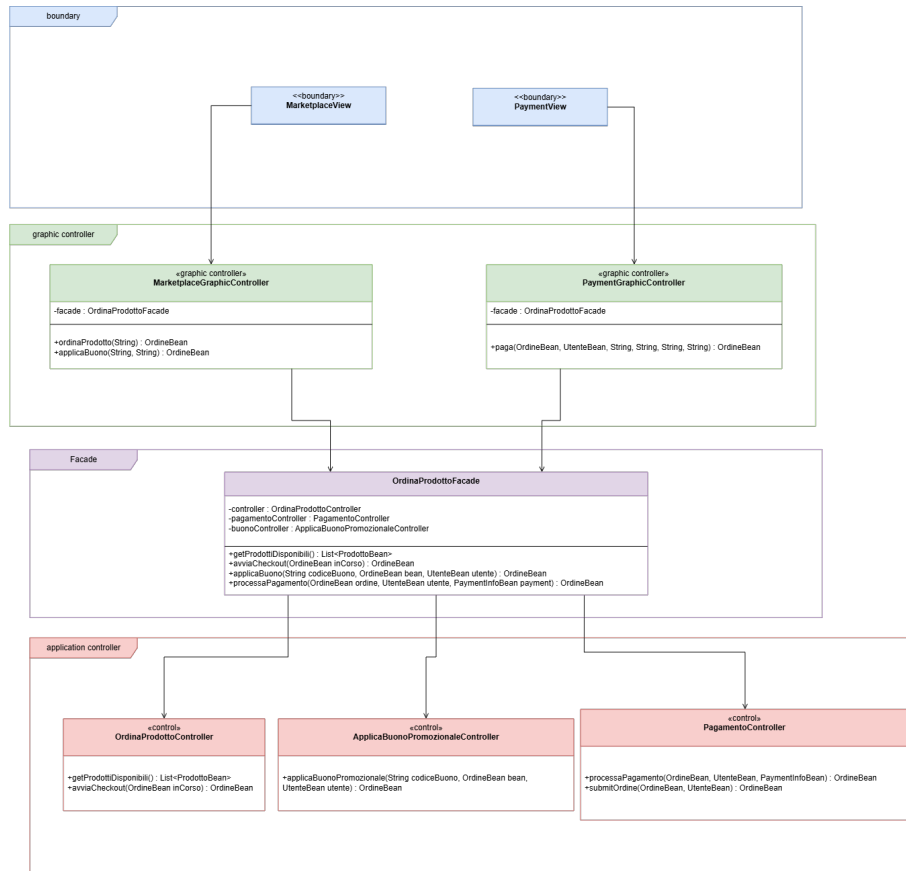


Figure 3.5: Facade of the UC-04 checkout.

`OrdinaProdottoFacade` exposes to the graphic controller a single interface for the checkout (`getProdottiDisponibili`, `avviaCheckout`, `applicaBuono`, `processaPagamento`) and hides the business subsystem, formed by the base controller and by the extension controllers (voucher 4a and payment 6/6a). The client (boundary or graphic controller) does not know the order in which the methods of the application controllers are called. The operations of the extensions (`applicaBuono`, `processaPagamento`) live only in their respective sub-controllers: the Facade invokes them directly and the base controller does not re-expose them (no useless pass-through / indirection).

3.2 Activity Diagram

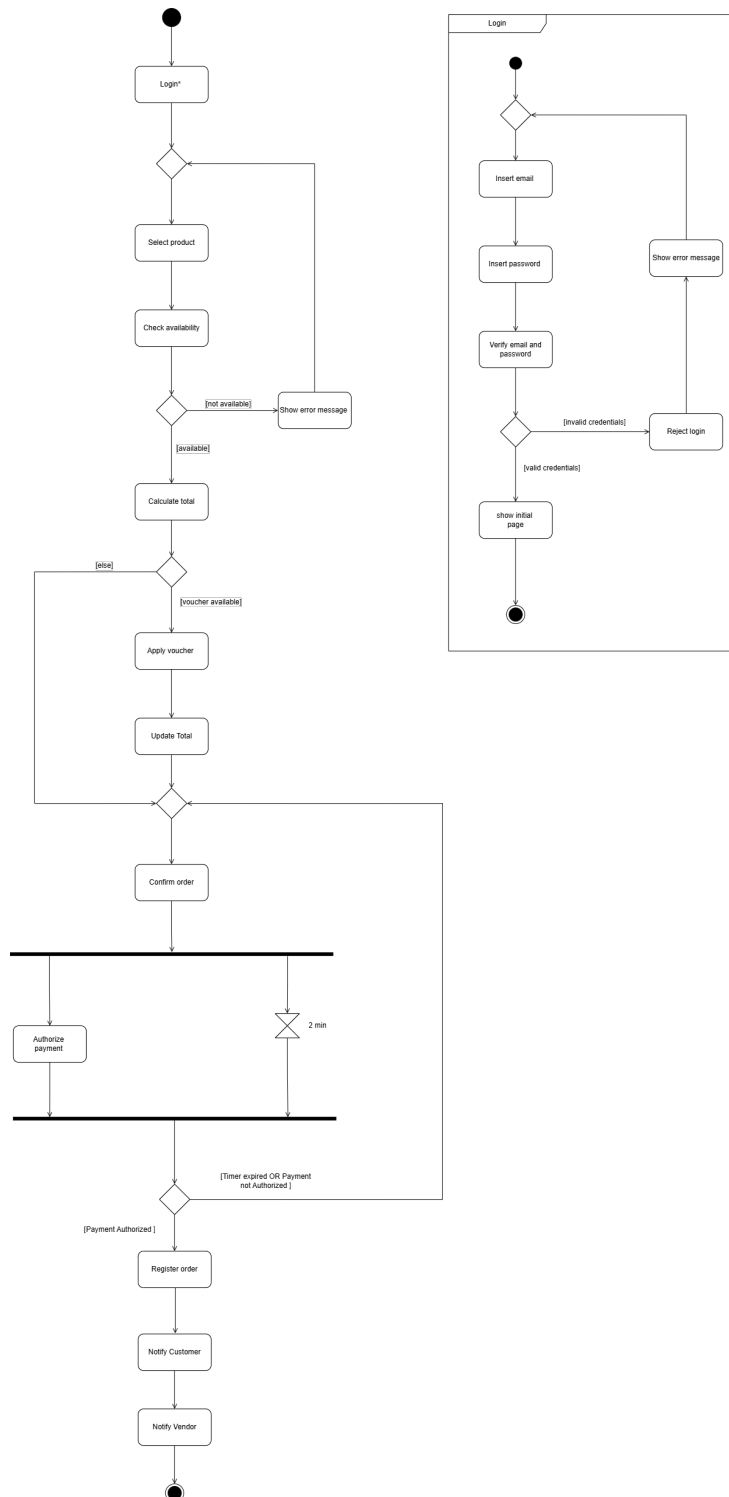


Figure 3.6: Activity diagram of UC-04: availability check, voucher application, payment authorization and sequential notification to buyer and vendor.

3.3 Sequence Diagram

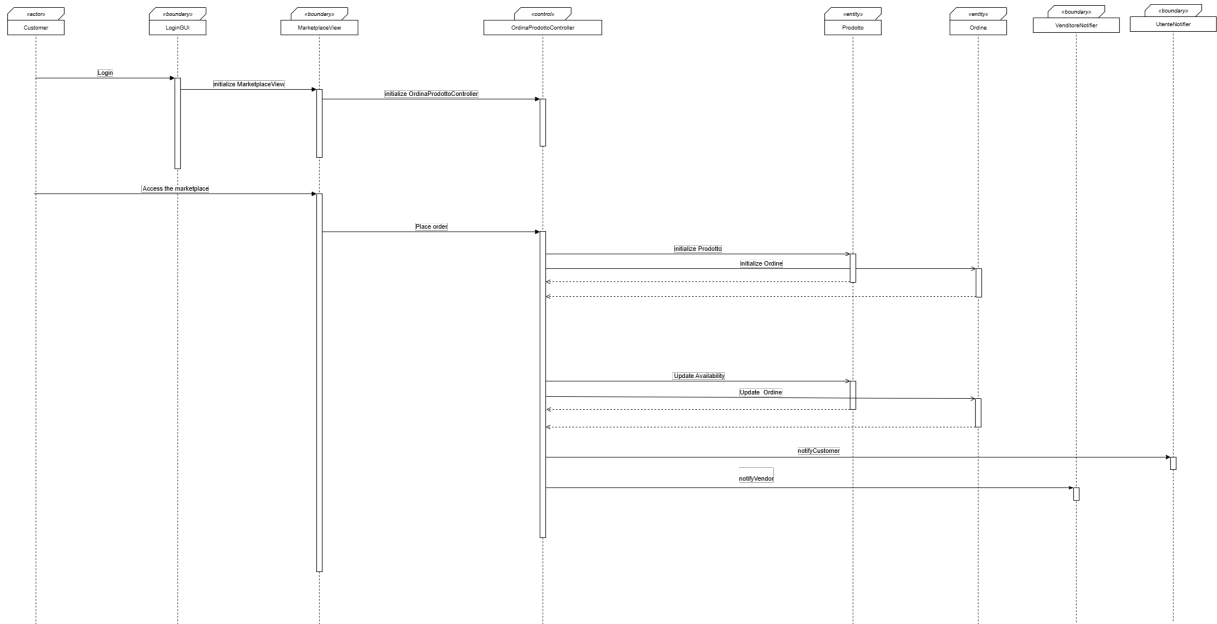


Figure 3.7: Sequence diagram of UC-04.

3.4 State Diagram

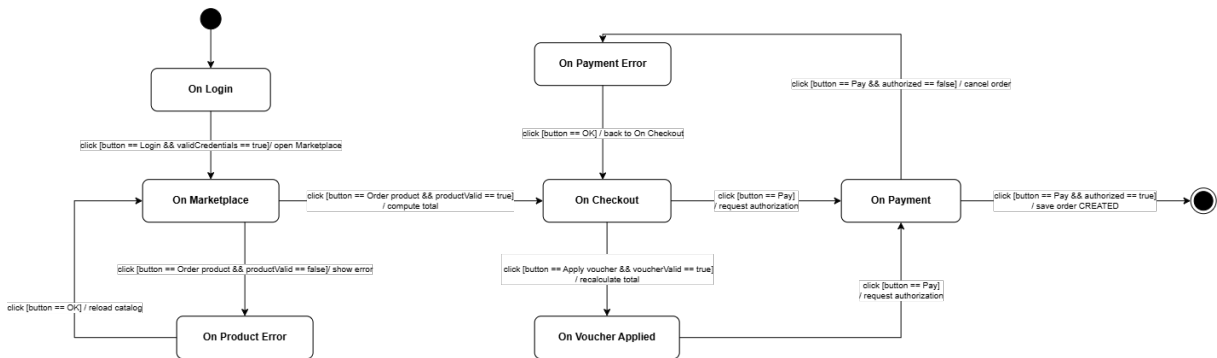


Figure 3.8: State diagram of UC-04 Order a Product: navigation and alternative flows 3a, 4a and 6a.

Chapter 4

Testing

The verification of the system is carried out with a **JUnit 5** unit test suite run by Maven, with coverage measured by JaCoCo (quality gate on line coverage). The verified behaviors are described below, grouped by feature.

Access to the restricted area. We tested the controller of the restricted area by verifying that an authenticated user correctly retrieves the data of their checkout and their session. The tests guarantee that the returned structures are never `null` and that the access to the restricted data is handled correctly for authenticated users.

Order creation and submission. We tested the ordering flow by verifying that a product is selected and that the order is created with the correct vendor. The tests guarantee that an order with a null user or product is rejected and that the stock availability is reduced at purchase time without negative stock.

Voucher discount (Strategy). We tested the discount logic by verifying that a valid voucher is applied to the order total. The tests guarantee that empty, expired or invalid codes are rejected without unhandled exceptions and that the discount calculation never produces inconsistent amounts.

Payment and authorization. We tested the payment subsystem by verifying both the authorized and the rejected branch. The tests guarantee that a rejected payment raises the correct application exception leaving the order unchanged, and that invalid card data (CVV, non-positive amount, missing order) are blocked without `NullPointerException`.

Authentication. We tested the authentication mechanism by verifying both valid and invalid credentials. The tests guarantee that a correct login populates the session and that wrong email or password are rejected, with the `logout` correctly clearing the session.

Order lifecycle (BR-04 state machine). We tested the order state machine by verifying the sequence `CREATED` $\rightarrow$ `CONFIRMED` $\rightarrow$ `IN_DELIVERY` $\rightarrow$ `DELIVERED`. The tests guarantee that an illegal transition is blocked by the invalid state exception.

Observer notification. We tested the Observer pattern by verifying that the publisher notifies the registered observers when an order is confirmed. The tests guarantee that the event (`OrdineEvent`) is delivered to the observers through the read-only DTO, never the domain entity.

Persistence (Demo and File System). We tested the persistence layers by verifying the create-read-update cycle of users, products and orders. The tests guarantee that the *Demo* (in-memory) and *File System* persistence keep the data consistently, including the retrieval of products by name.

Chapter 5

Exceptions

The error logic is a taxonomy (exception/) that is fully **checked**, forcing the explicit handling (throws / try/catch).

Exception	Role
CiboAmicoException (abstract)	Checked base, common fields
BusinessValidationException	Violated business rule
AutenticazioneException	Wrong credentials
InvalidStateTransitionException	Illegal state (BR-04)
DAOException	Persistence (file / DB)

The base `CiboAmicoException` extends `java.lang.Exception` (**checked**) and exposes four fields: `userMessage` (shown to the user), `technicalMessage` (for logs), `errorCode` and `timestamp`. The views read only `getUserMessage()`.

5.1 Wrapping

The DAOs wrap the low-level error (`SQLException`, `IOException`) in a `DAOException` with (message, cause): `getCause()` preserves the original cause, which never propagates to the boundaries.

5.2 Payment

`PaymentRejectedException` (pattern.payment) is checked: an unauthorized transaction, handled by extension 6a of UC-04.

5.3 Messages

Texts are centralized in `ExceptionMessagesEnum` (logs) and `UserErrorMessagesEnum` (user), without duplicated strings at the throw points.

Chapter 6

Persistence

Persistence is interchangeable at runtime (NFR-01): three mutually exclusive backends are selected through the **Abstract Factory** pattern (`DAOFactory` plus one concrete factory per mode).

6.1 DB persistence layer (MySQL/JDBC)

The DB persistence layer uses **MySQL** as a relational DBMS. Data management is delegated to specific DAO classes (e.g. `JDBCUserDAO`, `JDBCOrdineDAO`), which run SQL queries for the CRUD operations on the entities.

Main entities:

- **Users:** registered, with their roles (Customer and Vendor);
- **Orders:** transactional records with state, total and items;
- **Products:** marketplace items (catalog and inventory);
- **Vouchers:** discount codes (percentage or fixed amount).

Transactions are atomic (`ConnectionManager` uses `setAutoCommit(false)` with `commit/rollback` for the stock reduction).

Configuration: the system is configured through the `config.properties` file by setting `persistence_type = DB` and providing the required MySQL credentials.

6.2 FS persistence layer (File System)

The FS persistence layer stores the data on **JSON** files in the `data/` folder. Each entity is mapped to a dedicated file (e.g. `utenti.json`, `ordini.json`, `prodotti.json`, `inventario.json`).

Management: the `GsonConfig` class provides a consistent JSON serializer (helper for `LocalDate/LocalDateTime`) used by all the FS DAOs to guarantee the correct reading/writing of the records.

Configuration: the mode is activated by setting `persistence_type = FS` in the configuration file.

6.3 DEMO persistence layer (In-Memory)

The DEMO persistence layer keeps the data exclusively in the volatile memory (RAM), without permanent storage. The structures are populated with default values at startup and cleared at the end of the execution.

Use cases: it is designed for unit testing, rapid development and demonstrations that require no database configuration.

Configuration: the mode is activated by setting `persistence_type = DEMO` (default).

Chapter 7

Video and Links

7.1 GitHub Repository

<https://github.com/Damiano-o/ciboamico.git>

7.2 SonarCloud

<https://sonarcloud.io/organizations/ciboamico-ispw>

7.3 Demonstration video

To be added before the submission.